Activity 3.2 Random Forest

Objective(s):

This activity aims to perform classification using Random Forest

Intended Learning Outcomes (ILOs):

- Demonstrate how to build the model using Random Forest.
- Demonstrate how to evaluate the performance of the model.

Resources:

- Jupyter Notebook
- loan_data

Procedure:

We will use lending data from 2007-2010 and be trying to classify and predict whether or not the borrower paid back their loan in full.

Import the libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

Load the data and check the content of the dataframe using Pandas

```
loans= pd.read_csv('loan_data.csv')
loans.head()
```


```
{"summary":"{\n  \"name\": \"loans\",\n  \"rows\": 9578,\n
\"fields\": [\n    {\n      \"column\": \"credit.policy\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0,\n        \"min\": 0,\n        \"max\": 1,\n
\"num_unique_values\": 2,\n        \"samples\": [\n          0,\n
1\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"purpose\",\n      \"properties\": {\n        \"dtype\":
\"category\",\n        \"num_unique_values\": 7,\n        \"samples\":
[\n          \"debt_consolidation\",\n          \"credit_card\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"int.rate\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.026846987213382724,\n        \"min\": 0.06,\n        \"max\":
0.2164,\n        \"num_unique_values\": 249,\n        \"samples\": [\n
```

```
0.1367,\n          0.1496\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"installment\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 207.07130149985852,\n
\"min\": 15.67,\n        \"max\": 940.14,\n
\"num_unique_values\": 4788,\n        \"samples\": [\n
326.02,\n          182.39\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"log.annual.inc\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.6148127513545946,\n
\"min\": 7.547501683,\n        \"max\": 14.52835448,\n
\"num_unique_values\": 1987,\n        \"samples\": [\n
10.22557105,\n          11.19908503\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"dti\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 6.883969540539585,\n
\"min\": 0.0,\n        \"max\": 29.96,\n        \"num_unique_values\":
2529,\n        \"samples\": [\n          5.86,\n          21.4\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"fico\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\": 37,\n
\"min\": 612,\n        \"max\": 827,\n        \"num_unique_values\":
44,\n        \"samples\": [\n          647,\n          717\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"days.with.cr.line\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 2496.9303768113477,\n        \"min\":
178.9583333,\n        \"max\": 17639.95833,\n
\"num_unique_values\": 2687,\n        \"samples\": [\n
2432.0,\n          8669.958333\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"revol.bal\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 33756,\n        \"min\": 0,\n
\"max\": 1207359,\n        \"num_unique_values\": 7869,\n
\"samples\": [\n          11196,\n          462\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"revol.util\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
29.014416965641058,\n        \"min\": 0.0,\n        \"max\": 119.0,\n
\"num_unique_values\": 1035,\n        \"samples\": [\n          28.6,\
n          1.7\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"inq.last.6mths\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 2,\n        \"min\": 0,\n
\"max\": 33,\n        \"num_unique_values\": 28,\n        \"samples\":
[\n          33,\n          24\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"delinq.2yrs\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n
```

```
\"max\": 13,\n        \"num_unique_values\": 11,\n        \"samples\":
[\n          5,\n          0\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"pub.rec\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0,\n        \"min\": 0,\n
\"max\": 5,\n        \"num_unique_values\": 6,\n        \"samples\":
[\n          0,\n          1\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"not.fully.paid\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n
\"max\": 1,\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          1,\n          0\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"loans"}
```


Examine the data types.

```
loans.dtypes
```

```
credit.policy          int64
purpose               object
int.rate             float64
installment          float64
log.annual.inc       float64
dti                  float64
fico                   int64
days.with.cr.line    float64
revol.bal              int64
revol.util           float64
inq.last.6mths         int64
delinq.2yrs            int64
pub.rec                int64
not.fully.paid         int64
dtype: object
```

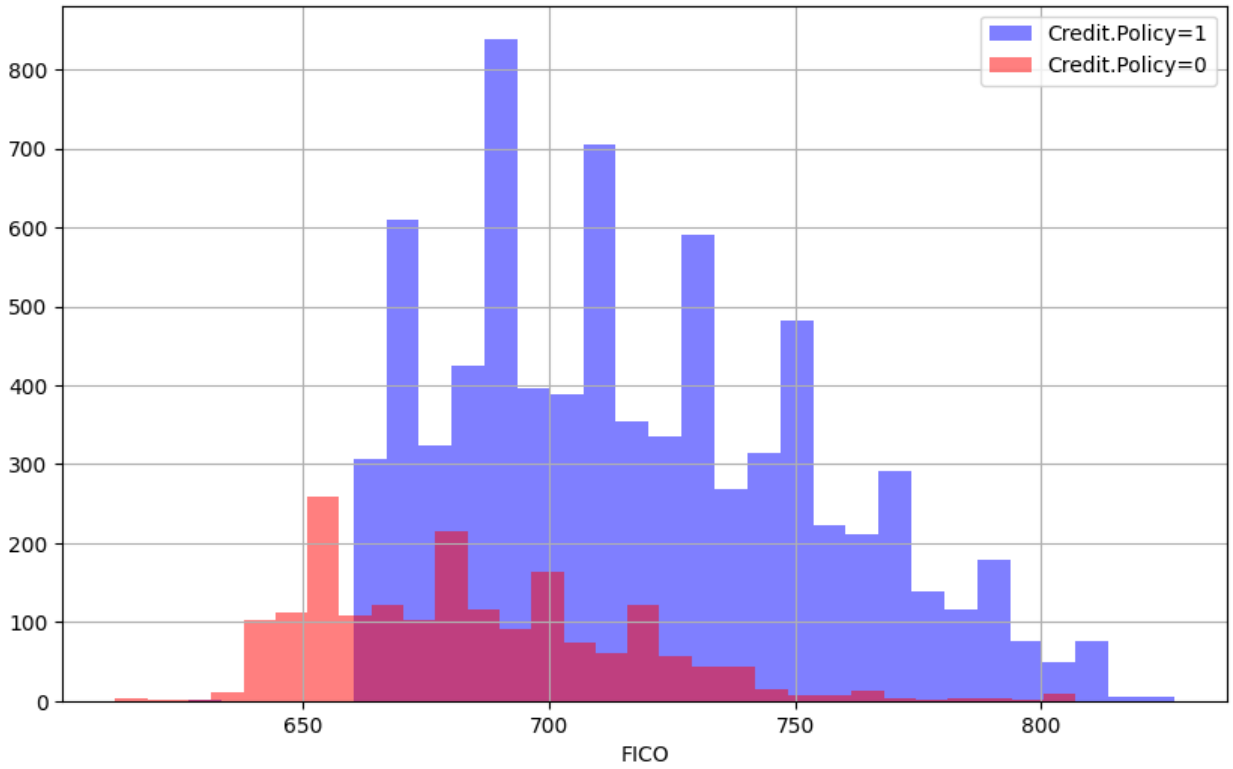
Create a histogram of two FICO distributions on top of each other, one for each credit.policy outcome.

```
plt.figure(figsize=(10,6))
loans[loans['credit.policy']==1]['fico'].hist(alpha=0.5,color='blue',

bins=30,label='Credit.Policy=1')
loans[loans['credit.policy']==0]['fico'].hist(alpha=0.5,color='red',

bins=30,label='Credit.Policy=0')
plt.legend()
plt.xlabel('FICO')
```

```
Text(0.5, 0, 'FICO')
```



Interpret the result of the graph

The graph shows the distribution of credit transactions grouped into two, including following and not following the credit policies. The distributions shows that most of the recorded transactions follow the credit policy, and populated the lower to higher FICO score ranges. Less records of transactions that do not follow the credit policy were seen, as well as populating the lower FICO score ranges. This simply indicates that transactions that follow the credit policy are more likely to be accepted for loan and to repay their loans on time.

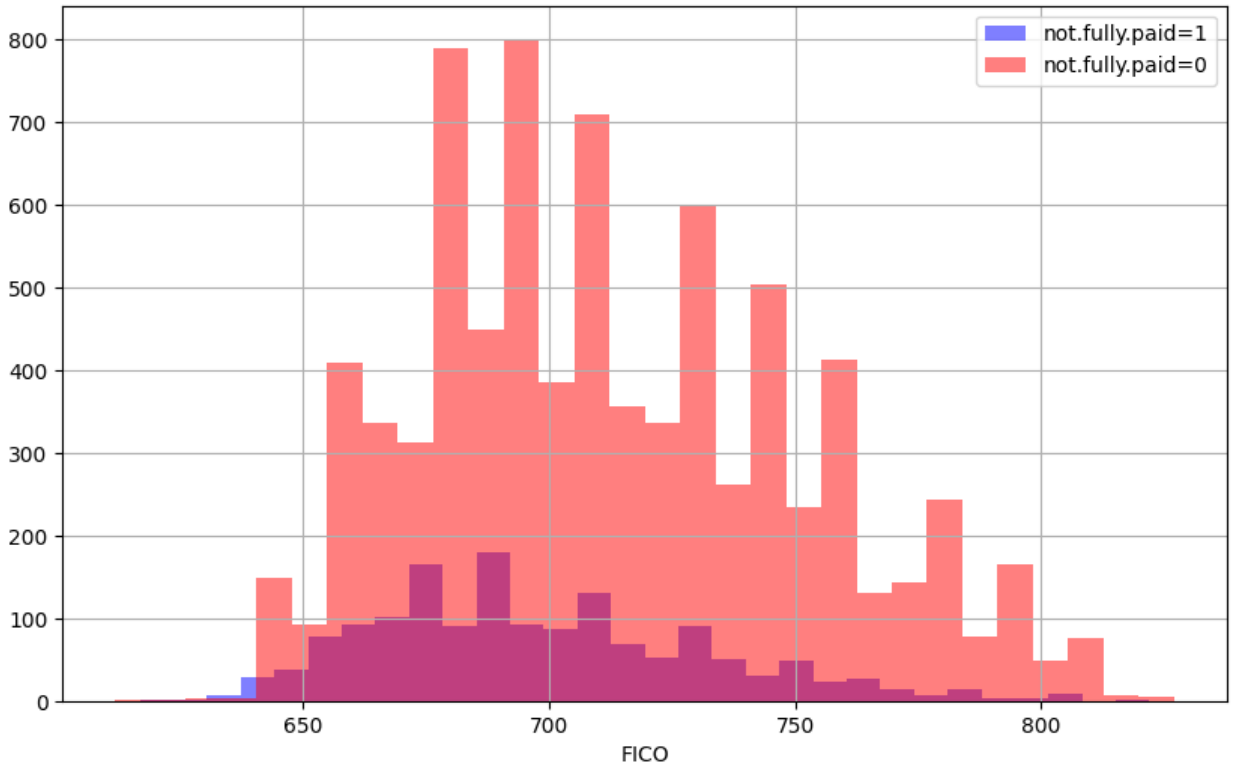
Create a similar figure, except this time select by the not.fully.paid column.

```
plt.figure(figsize=(10,6))
loans[loans['not.fully.paid']==1]['fico'].hist(alpha=0.5,color='blue',

bins=30,label='not.fully.paid=1')
loans[loans['not.fully.paid']==0]['fico'].hist(alpha=0.5,color='red',

bins=30,label='not.fully.paid=0')
plt.legend()
plt.xlabel('FICO')

Text(0.5, 0, 'FICO')
```



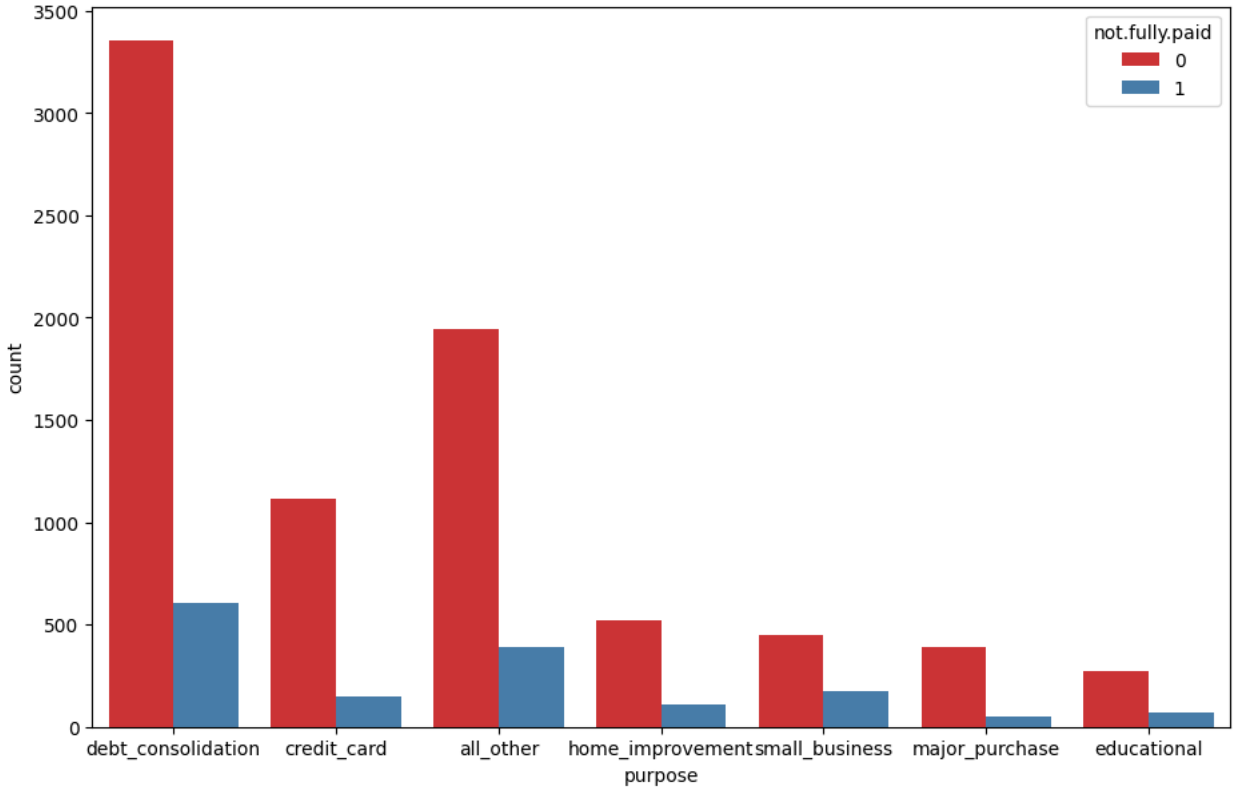
Interpret the result of the graph

The graph shows the distribution between lenders that are fully paid and not fully paid. Many lenders in the dataset are still not fully paid, and just a portion were fully paid. Lenders with FICO scores from 650 to 750 indicate higher distribution.

Create a countplot using seaborn showing the counts of loans by purpose, with the color hue defined by not.fully.paid.

```
plt.figure(figsize=(11,7))
sns.countplot(x='purpose',hue='not.fully.paid',data=loans,palette='Set
1')
```

```
<Axes: xlabel='purpose', ylabel='count'>
```



Interpret the result of the graph.

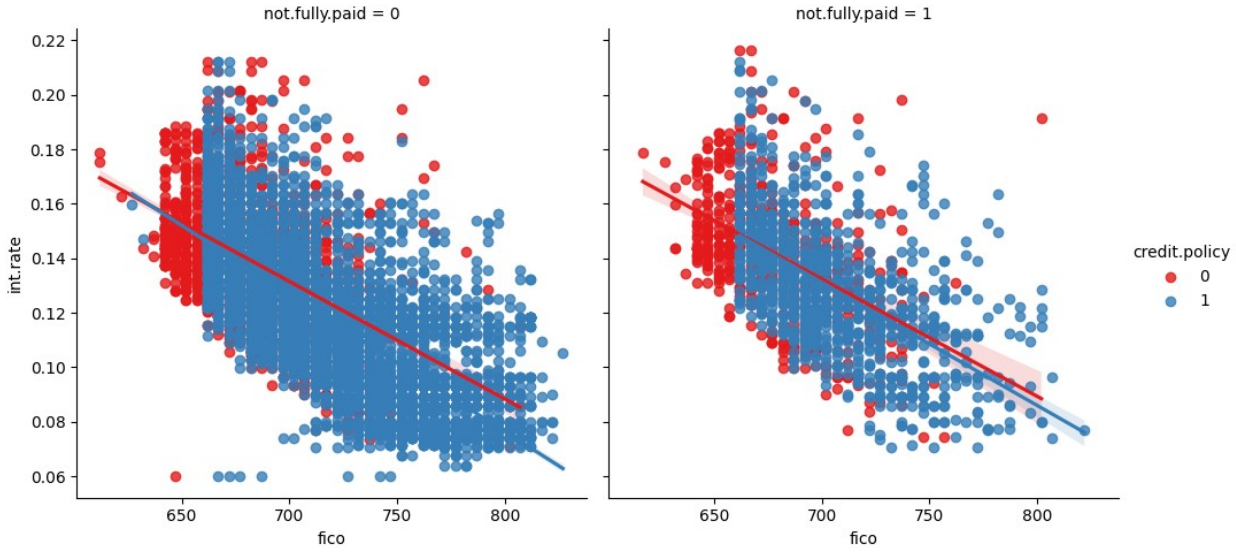
The bar graph shows the specific distribution of lenders that are fully paid and not fully paid, where it indicates the lending purpose. Lenders with debt consolidation indicate the highest count, followed by credit card and other purposes. Based on the definition of debt consolidation, it involves a lower interest rate, making it a popular choice for lenders.

Create the following lmplots to see if the trend differed between not.fully.paid and credit.policy.

```
plt.figure(figsize=(11,7))
sns.lmplot(y='int.rate',x='fico',data=loans,hue='credit.policy',
           col='not.fully.paid',palette='Set1')
```

```
<seaborn.axisgrid.FacetGrid at 0x7834d1f68690>
```

```
<Figure size 1100x700 with 0 Axes>
```



Interpret the result of the graph

The graph simply indicates that lenders, fully paid or not fully paid, are more likely to accept loans having interest rates from 12% to 18%. The lenders that are not fully paid also have FICO scores from 650-800, while fully paid lenders have FICO scores from 650-750.

The **purpose** column contains categorical value. Therefore, we need to transform the data using dummy variables. otice that the **purpose** column as categorical

```
cat_feats = ['purpose']

final_data = pd.get_dummies(loans,columns=cat_feats,drop_first=True)

final_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9578 entries, 0 to 9577
Data columns (total 19 columns):
 #   Column                     Non-Null Count  Dtype  
---  ------                     --------------  -----  
 0   credit.policy              9578 non-null   int64  
 1   int.rate                   9578 non-null   float64
 2   installment                9578 non-null   float64
 3   log.annual.inc             9578 non-null   float64
 4   dti                        9578 non-null   float64
 5   fico                       9578 non-null   int64  
 6   days.with.cr.line          9578 non-null   float64
 7   revol.bal                  9578 non-null   int64  
 8   revol.util                 9578 non-null   float64
 9   inq.last.6mths             9578 non-null   int64  
 10  delinq.2yrs                9578 non-null   int64  
 11  pub.rec                    9578 non-null   int64  
 12  not.fully.paid             9578 non-null   int64  
```

```
 13  purpose_credit_card          9578 non-null   bool
 14  purpose_debt_consolidation   9578 non-null   bool
 15  purpose_educational          9578 non-null   bool
 16  purpose_home_improvement     9578 non-null   bool
 17  purpose_major_purchase       9578 non-null   bool
 18  purpose_small_business       9578 non-null   bool
dtypes: bool(6), float64(6), int64(7)
memory usage: 1.0 MB
```

Split the data into a training set and a testing set

```
from sklearn.model_selection import train_test_split

X = final_data.drop('not.fully.paid',axis=1)
y = final_data['not.fully.paid']
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.30, random_state=101)
```

Create an instance of the RandomForestClassifier class and fit it to our training data

```
from sklearn.ensemble import RandomForestClassifier

rfc = RandomForestClassifier(n_estimators=600)

rfc.fit(X_train,y_train)

RandomForestClassifier(n_estimators=600)
```

What is n_estimators?

The n_estimators argument in RandomForestClassifier indicates the number of decision trees that will be made from the ensemble training. It is important to indicate the right amount of estimators to avoid overfitting.

Predict the class of not.fully.paid for the X_test data.

```
predictions = rfc.predict(X_test)
```

Create a classification report from the results

```
from sklearn.metrics import
classification_report,confusion_matrix,accuracy_score, roc_auc_score

print(classification_report(y_test,predictions))

              precision    recall  f1-score   support

           0       0.85      1.00      0.92      2431
           1       0.47      0.02      0.04       443
```

```
    accuracy                           0.85      2874
   macro avg       0.66      0.51      0.48      2874
weighted avg       0.79      0.85      0.78      2874
```

Show the Confusion Matrix for the predictions.

```
print(confusion_matrix(y_test,predictions))
```

```
[[2421   10]
 [ 434    9]]
```

```
#evaluate the performance using accuracy score
print(accuracy_score(y_test, predictions))
```

```
0.8455114822546973
```

```
print(roc_auc_score(y_test, predictions))
```

```
0.5081012467813689
```

Interpret the result of the classification error, confusion matrix , accuracy score and roc_auc_score

The results presented the model accuracy of classifying the 2 main classes in the dataset: lenders that are fully paid and not fully paid. Fully paid lenders were predicted with high precision (0.85), perfect recall (1.00), and a high F1-score (0.92). This means it is great at identifying loans that were fully paid. Not fully paid lenders had poor performance, with low precision (0.47), extremely low recall (0.02), and a very low F1-score (0.04). This indicates that the model struggles significantly to identify loans that were not fully paid, catching only 2% of them. The overall accuracy is 0.85 (85%). While seemingly high, this can be considered as misleading due to the significant class imbalance and unequal performance on predicting both classes.

The Confusion Matrix indicated that the model correctly predicted 2421 loans as 'fully paid', incorrectly predicted 10 loans as 'fully paid' when they were actually 'not fully paid', incorrectly predicted 434 loans as 'not fully paid' when they were actually 'fully paid', and correctly predicted only 9 loans as 'not fully paid'.

The ROC AUC score is approximately 0.5081.This is a strong indicator that the model is not effective at predicting which loans will not be fully paid.

Supplementary Activity:

- Choose your own dataset
- Import the dataset
- Determine the number of datapoints, columns and data types
- Remove unneccesary columns
- Do data cleaning such as removing empty values(NaN), replacing missing data .
- Perform descriptive statistics such as mean, median and mode

- Perform data visualization
- Build the model using Ranfom Forest
- Evaluate the model using classification report, accuracy , confusion matrix and roc_auc_score
- Change the n_estimators from 100 to 1000, increment by 100.
- Create a graph to compare the accuracy based from n_estimators

Dataset: Penguin Species Dataset

Importing dataset

```
# Importing dataset
df = pd.read_csv("penguins.csv")
df.head(5)
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 1000,\n  \"fields\":
[\n    {\n      \"column\": \"species\",\n      \"properties\": {\n
\"dtype\": \"category\",\n        \"num_unique_values\": 3,\n
\"samples\": [\n          \"Adelie\",\n          \"Gentoo\",\n
\"Chinstrap\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"island\",\n      \"properties\": {\n        \"dtype\":
\"category\",\n        \"num_unique_values\": 3,\n        \"samples\":
[\n          \"Torgersen\",\n          \"Biscoe\",\n
\"Dream\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"culmen_length_mm\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 5.309985167434392,\n        \"min\":
32.4,\n        \"max\": 56.0,\n        \"num_unique_values\": 206,\n
\"samples\": [\n          37.7,\n          50.5,\n          36.9\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"culmen_depth_mm\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.7790570332759408,\n        \"min\": 13.0,\n        \"max\": 21.4,\n
\"num_unique_values\": 80,\n        \"samples\": [\n          13.8,\n
18.4,\n          19.1\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"flipper_length_mm\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 13.17166038094355,\n        \"min\":
175.0,\n        \"max\": 235.0,\n        \"num_unique_values\": 58,\n
\"samples\": [\n          195.0,\n          189.0,\n          180.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"body_mass_g\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
705.3347935976439,\n        \"min\": 2830.0,\n        \"max\":
6317.0,\n        \"num_unique_values\": 786,\n        \"samples\": [\n
3714.0,\n          3352.0,\n          3454.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"sex\",\n      \"properties\": {\n
\"dtype\": \"category\",\n        \"num_unique_values\": 2,\n
```

```
\"samples\": [\n          \"Male\",\n          \"Female\"\n        ],\
n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    }\n  ]\n}","type":"dataframe","variable_name":"df"}
```

```
# Data points
print(f"Data Points: {df.shape[0]}\nColumns: {df.shape[1]}")
```

```
Data Points: 1000
Columns: 7
```

```
# Info
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 7 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   species            1000 non-null   object 
 1   island             1000 non-null   object 
 2   culmen_length_mm   990 non-null    float64
 3   culmen_depth_mm    990 non-null    float64
 4   flipper_length_mm  990 non-null    float64
 5   body_mass_g        990 non-null    float64
 6   sex                990 non-null    object 
dtypes: float64(4), object(3)
memory usage: 54.8+ KB
```

```
# Data types
df.dtypes
```

```
species               object
island                object
culmen_length_mm     float64
culmen_depth_mm      float64
flipper_length_mm    float64
body_mass_g          float64
sex                   object
dtype: object
```

```
# Describe
df.describe()
```

```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 8,\n  \"fields\": [\n
{\n      \"column\": \"culmen_length_mm\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 336.7891037609659,\n
\"min\": 5.309985167434392,\n        \"max\": 990.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
43.817777777777785,\n          43.8,\n          990.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"culmen_depth_mm\",\n
```

\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
344.72465376316666,\n   \"min\": 1.7790570332759408,\n
\"max\": 990.0,\n   \"num_unique_values\": 8,\n
\"samples\": [\n   17.335555555555555,\n   17.8,\n
990.0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"flipper_length_mm\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 296.3774700965604,\n   \"min\":
13.17166038094355,\n   \"max\": 990.0,\n
\"num_unique_values\": 8,\n   \"samples\": [\n
199.22424242424242,\n   195.0,\n   990.0\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"body_mass_g\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
1854.287592761864,\n   \"min\": 705.3347935976439,\n
\"max\": 6317.0,\n   \"num_unique_values\": 8,\n
\"samples\": [\n   4103.311111111111,\n   3882.0,\n
990.0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe"}

Data Cleaning

```
# Create copy
df_clean = df.copy()

# Clean missing rows
df_clean = df.dropna()

df_clean.shape
```

```
(951, 7)
```

```
# Get numerical columns
dfc_numeric = df_clean.select_dtypes(include='number')

# Mean
dfc_numeric.mean()
```

```
culmen_length_mm       43.825026
culmen_depth_mm        17.324711
flipper_length_mm     199.191377
body_mass_g          4096.220820
dtype: float64
```

```
# Median
dfc_numeric.median()
```

```
culmen_length_mm       43.9
culmen_depth_mm        17.8
flipper_length_mm     195.0
```

```
body_mass_g           3880.0
dtype: float64
```

```
# Mode
dfc_numeric.mode()
```


```
{"summary":"{\n  \"name\": \"dfc_numeric\",\n  \"rows\": 4,\n
\"fields\": [\n    {\n      \"column\": \"culmen_length_mm\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 38.3,\n        \"max\": 38.3,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          38.3\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"culmen_depth_mm\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 18.2,\n        \"max\": 18.2,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          18.2\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"flipper_length_mm\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
null,\n        \"min\": 189.0,\n        \"max\": 189.0,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          189.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"body_mass_g\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
198.1302181226613,\n        \"min\": 3318.0,\n        \"max\":
3755.0,\n        \"num_unique_values\": 4,\n        \"samples\": [\n
3540.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```

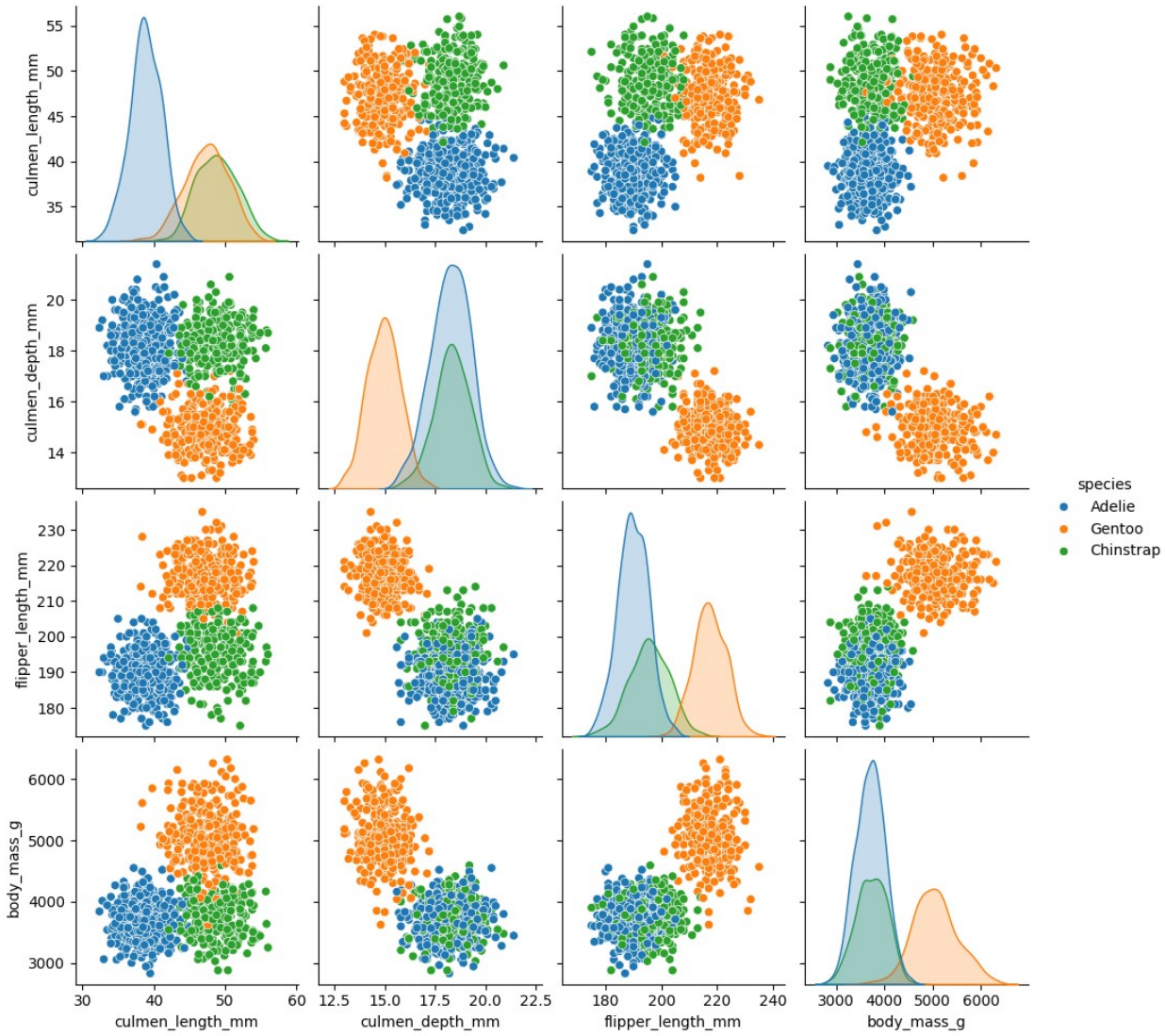

```
# Clean df
df_clean.head()
```


```
{"summary":"{\n  \"name\": \"df_clean\",\n  \"rows\": 951,\n
\"fields\": [\n    {\n      \"column\": \"species\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 3,\n        \"samples\": [\n
\"Adelie\",\n          \"Gentoo\",\n          \"Chinstrap\"\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"island\",\n      \"properties\": {\n        \"dtype\":
\"category\",\n        \"num_unique_values\": 3,\n        \"samples\":
[\n          \"Torgersen\",\n          \"Biscoe\",\n
\"Dream\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"culmen_length_mm\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 5.326905329930786,\n        \"min\":
32.4,\n        \"max\": 56.0,\n        \"num_unique_values\": 204,\n
\"samples\": [\n          37.7,\n          50.5,\n          52.9\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"culmen_depth_mm\",\n
```

```
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.7705037331010263,\n        \"min\": 13.0,\n        \"max\": 21.4,\n
\"num_unique_values\": 79,\n        \"samples\": [\n          16.1,\n
18.4,\n          19.1\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"flipper_length_mm\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 13.183449324835415,\n        \"min\":
175.0,\n        \"max\": 235.0,\n        \"num_unique_values\": 58,\n
\"samples\": [\n          195.0,\n          189.0,\n          180.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"body_mass_g\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
703.6957802823009,\n        \"min\": 2830.0,\n        \"max\":
6317.0,\n        \"num_unique_values\": 763,\n        \"samples\": [\n
4392.0,\n          4153.0,\n          4231.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"sex\",\n      \"properties\": {\n
\"dtype\": \"category\",\n        \"num_unique_values\": 2,\n
\"samples\": [\n          \"Male\",\n          \"Female\"\n        ],\
n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    }\n  ]\n}","type":"dataframe","variable_name":"df_clean"}
```


Data visualization

```
sns.pairplot(data=df_clean, hue="species")
plt.show()
```




```
# convert species to numerical
df_clean['species'] = df_clean['species'].map({'Adelie': 0,
'Chinstrap': 1, 'Gentoo': 2})
```

```
/tmp/ipykernel_2645/2668828828.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
  df_clean['species'] = df_clean['species'].map({'Adelie': 0,
'Chinstrap': 1, 'Gentoo': 2})
```

```
X = df_clean.drop(['species', 'island', 'sex'],axis=1)
y = df_clean['species']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.30, random_state=101)
```

Create an instance of the RandomForestClassifier class and fit it to our training data

```
rfc = RandomForestClassifier(n_estimators=600)

rfc.fit(X_train,y_train)

RandomForestClassifier(n_estimators=600)
```

What is n_estimators?

Type your answer here

Predict the class of not.fully.paid for the X_test data.

```
predictions = rfc.predict(X_test)
```

Create a classification report from the results

```
from sklearn.metrics import
classification_report,confusion_matrix,accuracy_score, roc_auc_score

print(classification_report(y_test,predictions))
```

```
              precision    recall  f1-score   support

           0       0.99      0.99      0.99       120
           1       0.99      0.99      0.99        77
           2       1.00      1.00      1.00        89

    accuracy                           0.99       286
   macro avg       0.99      0.99      0.99       286
weighted avg       0.99      0.99      0.99       286
```

Show the Confusion Matrix for the predictions.

```
print(confusion_matrix(y_test,predictions))

[[119   1   0]
 [  1  76   0]
 [  0   0  89]]

#evaluate the performance using accuracy score
print(accuracy_score(y_test, predictions))

0.993006993006993
```

```
print(roc_auc_score(y_test, rfc.predict_proba(X_test),
multi_class='ovr'))
```

```
0.9999625534595559
```

Conclusion

The activity demonstrated the application of the Random Forest Classifier model to a loan dataset. The data was initially analyzed to learn more about the data. A Random Forest Classifier was then applied to it. Upon predicting on the test set, the model yielded a high overall accuracy of about 85%. However, a detailed evaluation using the classification report, confusion matrix, and ROC AUC score revealed significant limitations: while performing well on 'fully paid' loans, it struggled severely with the minority class of 'not fully paid' loans.

The supplementary activity demonstated the usage of Random Forest Classifier to the penguin species dataset. The model achieved a high accuracy of 99.3%, demonstrating excellent performance in classifying the different penguin species. The precision, recall, and F1-score for each species were also very high, ranging from 99% to 100%,. The ROC AUC score further supports this, with a value of 0.9999. This indicates how the Random Forest Classifier was appropriate to the dataset, showing robust and reliable results.